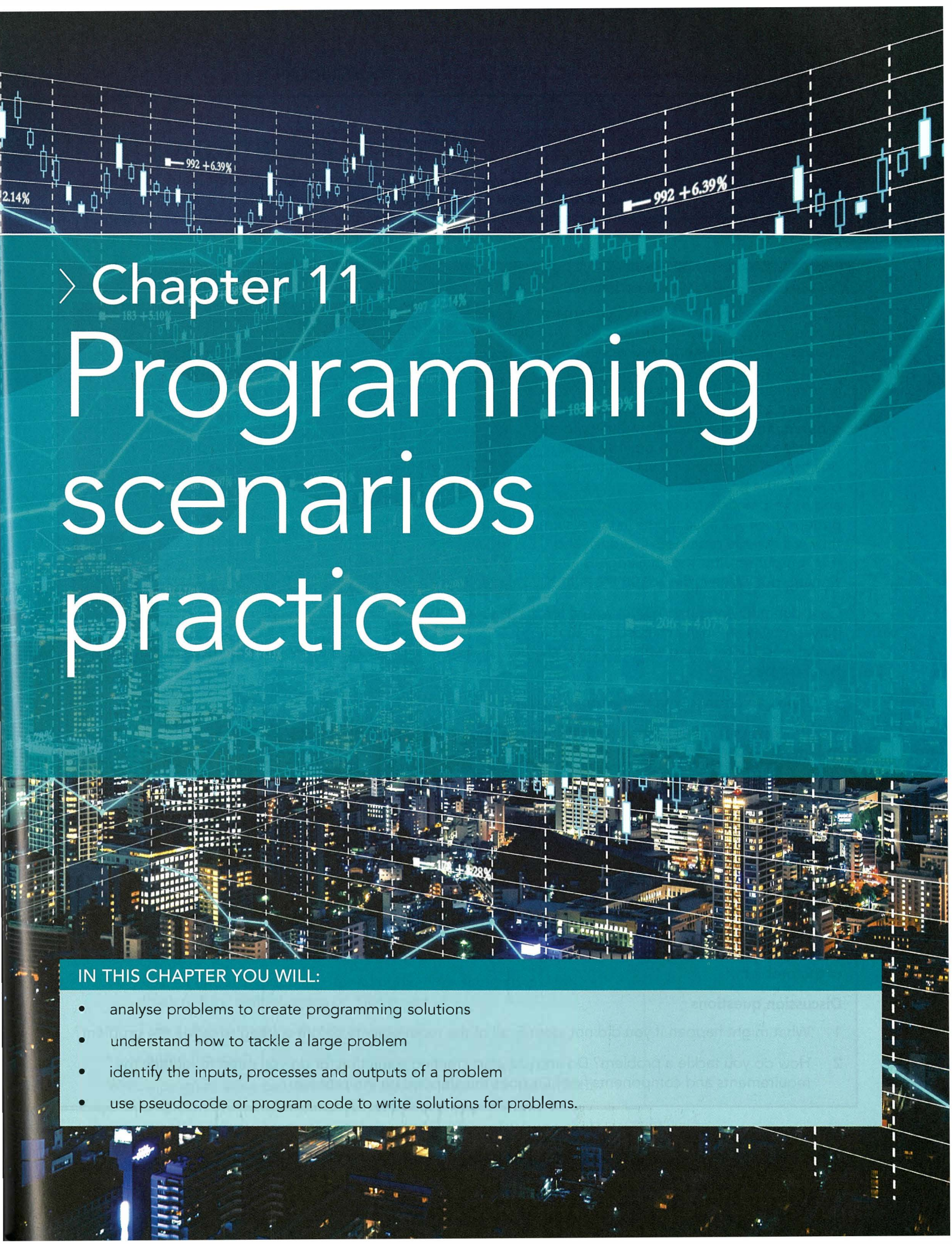


Chapter 11

Programming scenarios practice

IN THIS CHAPTER YOU WILL:

- analyse problems to create programming solutions
- understand how to tackle a large problem
- identify the inputs, processes and outputs of a problem
- use pseudocode or program code to write solutions for problems.

GETTING STARTED

Work in pairs to discuss how you tackle a programming problem. What do you do first? How do you check that you have met all the requirements? What features do you add to improve the readability of your program?

PLANNING A PROGRAM

Programmers have to tackle programming problems every day. They have to analyse the requirements and work out how to write a program to meet those requirements, in the same way that you will need to. The only difference is that programmers have been doing this process for a long time, they have practised and practised. This is the best way to tackle how to write programs; practice, practice and more practice. Most programs follow a pattern, they will have inputs in the system, they will include processes (actions), and then output results. The challenge is working out which order to put these in.

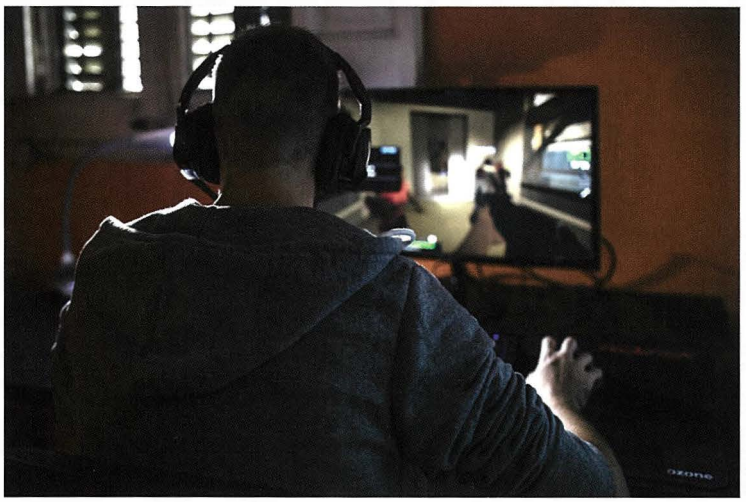


Figure 11.1: A person playing a computer game

Consider a 3D computer game, where the player moves an online character around a virtual world, interacting with objects and other characters. This system has inputs, processes and outputs. For example:

- Input: an arrow key from the user (up, down, left, right).
- Process: change the characters position depending on the key pressed.
- Output: the character in the new position.

From this analysis a program can be written to perform these actions.

The larger the problem, the more likely a design will need to be created first. This design will identify all of the requirements of the problem, and begin to identify the inputs, processes and outputs. For some programs there could be thousands (or more) inputs, processes and outputs. They all need to be identified to make sure the design covers all the requirements.

The only difference between the process with a large problem, and the problem you will be tackling is the size. You are only going to need to identify a small number of inputs, processes and outputs and you are working on your own. A large program could have dozens of programmers, each working on their own part of the problem.

Discussion questions

1 What might happen if you did not identify all of the requirements before starting to tackle the problem?

2 How do you tackle a problem? Do you just start creating a solution, or do you need to identify the requirements and components first? Or does this depend on the problem?

11.1 Programming scenario

In the Paper on Algorithms, Programming and Logic, you will be given a scenario that you need to write an algorithm for to solve the problem. It is important that you think about the problem and plan before you start writing a solution. This is especially important to make sure you don't miss anything that is in the question.

There are then two ways that you could tackle this problem. Which you choose depends on you and which you prefer.

The first method is identifying the inputs, processes and outputs. The second method is practically doing the programming task and identifying the steps this way.

COMPUTER SCIENCE IN CONTEXT

Consider your favourite computer game. At some point a programmer had to identify the inputs, processes and outputs in the game.

ACTIVITY 11.1

Write down some of the inputs, processes and outputs for your favourite computer game.

Peer Assessment

Compare your game with a partner's. Can you think of any more inputs, processes or outputs for their game?

11.2 Method 1: Identifying the inputs, processes and outputs

Identify key features

First read the problem **twice** before starting to work out what you need to do.

Example – calculator:

A program needs writing to act as a calculator. The program needs to:

- Take one integer value from the user.
- Take an operator as input; + for addition, − for subtraction, / for division, * for multiplication or ^ for power of.
- Take a second integer value from the user.
- Output the result.

Write a program using pseudocode or program code to solve the problem.

Once you have read it twice, start to identify the key features; the inputs, processes and outputs. You could do this using a table, e.g.

Input	Process	Output

Identify the inputs from the scenario:

Input	Process	Output
First integer value. Operator. Second integer operator.		

Then identify the outputs:

Input	Process	Output
First integer value. Operator. Second integer operator.		Result from the calculation.

Then you need to work out how to get from the input to the output. This could be a description, program statements, or anything that helps you to write down what happens.

If you get stuck here, try doing what the program needs. In the case of the calculator, write down the first integer value (e.g. 3), write down the operator (e.g. +), write down the second integer value (1). Then what do you do with this values? Because the operator is + you will do 3 + 1. That is your process.

Input	Process	Output
First integer value. Operator. Second integer operator.	If + is entered add the values. If – is entered subtract the values. If * is entered multiply the values. If / is entered divide the values. if ^ is entered work out the power of.	Result from the calculation.

ACTIVITY 11.2

Open a program you have already written. Identify the inputs, processes and outputs for this program.

REFLECTION

How did you investigate the inputs, processes and outputs for Activity 11.2? Did you work through each line of code at a time, or did you just focus on the elements you were looking for?

11.3 Writing the code

Now you know the inputs, processes and outputs, you need to work out the order. Usually the inputs come first, and they are a good place to start. Even if can't identify the processes yet, always do the inputs – they will help to get your started.

The question will likely say pseudocode or program code, this is to allow you the freedom of what to use. It does not mean that your code must be syntactically perfect, but it must use code structures, for example:

`For x ← 1 TO 10` is code like, but,
`starting with 1 loop until 10` is not code-like, this is English statements not code statements.

COMPUTER SCIENCE IN CONTEXT

Pseudocode is supposed to be a language neutral way of designing code. This means that any programmer can take the pseudocode you have written, and write it using their programming language of choice.

TIP

The pseudocode conventions used in this chapter appear exactly as you will see in your exams.

Inputs

It is always best to include an output before the input, to tell the user what to enter. This can be done in two statements, e.g.

```
OUTPUT "Enter a number"
INPUT number
```

or as one statement, e.g.

```
number ← INPUT "Enter a number"
```

Whichever way you choose, make sure you are inputting a value into a variable. In the examples above the variable is `number`. If you use this statement:

```
INPUT "Enter a number"
```

then the number will be input and it won't be stored anywhere, so you can't use it.

Example – calculator:

Write code for the inputs.

```
Number1 ← INPUT "Enter a number"
Operator ← INPUT "Enter an operator"
Number2 ← INPUT "Enter a number"
```

Validation

Now you have your inputs, consider if there are limits on what should be entered. Look at each input in turn and write down what is, and isn't allowed.

Example – calculator:

- First integer value – must be a whole number.
- Operator – must be + − / * or ^.
- Second integer operator – must be a whole number.

There are different ways that you could validate the inputs, e.g.

- Use an if statement and only allow the rest of the program to run if the inputs it valid.
- Use a loop to continue asking for the input until it is valid.

Example – calculator:

```
REPEAT
    Number1 ← INPUT "Enter a number"
UNTIL Number1.isInteger()

REPEAT
    Operator ← INPUT "Enter an operator"
UNTIL Operator = "+" OR Operator = "-" OR Operator = "*" OR Operator = "/"
    OR Operator = "^"

REPEAT
    Number2 ← INPUT "Enter a number"
UNTIL Number2.isInteger()
```

NB the statement `.isInteger` is not language specific, but as a pseudocode statement it shows that you are looking to check if the value is an integer and only allow the input if it is. You might have one for the language you choose, but it is better to use this than a statement such as: `until the number is an integer.`

Processes

Revisit your design table and take the processes one at a time and work out whether they need sequence, selection and/or iteration.

Example – calculator:

Process		Construct
1	If + is entered add the values.	selection
2	If − is entered subtract the values.	selection
3	If * is entered multiply the values.	selection
4	If / is entered divide the values.	selection
5	If ^ is entered work out the power of.	selection

Process 1. If + is entered add the values.

```
IF Operator = "+"
    THEN
        Result ← Number1 + Number2
```

```
ELSEIF Operator = "-"
    THEN
        Result ← Number1 - Number2
ELSEIF Operator = "*"
    THEN
        Result ← Number1 * Number2
ELSEIF Operator = "/"
    THEN
        Result ← Number1 / Number2
    ELSE
        Result ← Number1 ^ Number2
ENDIF
```

Outputs

Once you have written your processes, work out where your outputs should be. Make sure you output using an appropriate message, for example, what the value is showing.

Example – calculator:

Input	Process	Output
First integer value. Operator. Second integer operator.	If + is entered add the values. If – is entered subtract the values. If * is entered multiply the values. If / is entered divide the values. if ^ is entered work out the power of.	Result from the calculation.

The output is:

```
OUTPUT "The result is: " , Result
```

or

```
OUTPUT Number1 , Operator , Number2 , " = " , Result
```

Example – calculator, full code:

```
REPEAT
    Number1 ← INPUT "Enter a number"
UNTIL Number1.isInteger()

REPEAT
    Operator ← INPUT "Enter an operator"
UNTIL Operator = "+" OR Operator = "-" OR Operator = "*"
    OR Operator = "/" OR Operator = "^"

REPEAT
    Number1 ← INPUT "Enter a number"
UNTIL Number1.isInteger()

IF Operator = "+"
    THEN
        Result ← Number1 + Number2
```

TIP

When outputting data, it is good practice to include a message to say what the output is. This is the same in a small pseudocode problem as in a large program being created by many people. If you run a program and it outputs a number without a message – how do you know what that number means?

```
ELSEIF Operator = "-"
    THEN
        Result ← Number1 - Number2
ELSEIF Operator = "*"
    THEN
        Result ← Number1 * Number2
ELSEIF Operator = "/"
    THEN
        Result ← Number1 / Number2
    ELSE
        Result ← Number1 ^ Number2
ENDIF
OUTPUT "The result is: ", Result
```

Adding comments

You should always add comments to your code to explain what you have done. Comments are statements in your program that will not be run. A comment will always start with a specific symbol to state that this is where the comment starts, for example, */, # or //.

Comments are used to explain sections of code, not every single statement.

Example – calculator:

```
// input 2 numbers and operator
REPEAT
    Number1 ← INPUT "Enter a number"
UNTIL Number1.IsInteger()

REPEAT
    Operator ← INPUT "Enter an operator"
UNTIL Operator = "+" OR Operator = "-" OR Operator = "*" OR Operator = "/"
    OR Operator = "^"

REPEAT
    Number1 ← INPUT "Enter a number"
UNTIL Number1.IsInteger()
//check operator and perform calculation

IF Operator = "+"
    THEN
        Result ← Number1 + Number2
ELSEIF Operator = "-"
    THEN
        Result ← Number1 - Number2
ELSEIF Operator = "*"
    THEN
        Result ← Number1 * Number2
ELSEIF Operator = "/"
    THEN
        Result ← Number1 / Number2
    ELSE
        Result ← Number1 ^ Number2
```

```
ENDIF
//output the result
OUTPUT "The result is: ", Result
```

Final checks

Once you have finished your algorithm perform the following checks:

1. Have you stored all of your inputs in variables?
2. Have you validated inputs where possible?
3. Have you performed all of the processes?
4. Have you outputted the required values?
5. Have you included appropriate messages in your outputs?
6. Have you added comments?
7. Do all your variables have appropriate identifiers?
8. Return to the scenario and tick every statement that your program performs. If you don't tick a statement then you need to add that to your program.

ACTIVITY 11.3

Read the calculator pseudocode solution above and complete the final checks on the algorithm. Tick each on in turn to make sure it has been covered in the solution.

SKILLS FOCUS 11.1

TACKLING A PROBLEM

When you are given a problem to write a computer program for, you need to know how to approach it to create a solution. In this Skills Focus you will be walked through the production of a solution by analysing the problem, identifying the components and writing a solution that covers all of the requirements. You can then apply the same methodology to other problems to write a computer program to solve the problem.

A computer game asks the user to enter a 5-letter word and then stores each letter in a different index in the array, for example:

'house' is input:

Index	0	1	2	3	4
Letter	h	o	u	s	e

A second user then has 10 guesses to try and work out what the word is. They have to enter one letter at a time, and the program either outputs the position that letter is in (e.g. 'h' is in 0), or that the letter is not in the word.

After guessing a letter the user gets a free guess at the word. If they get it correct they win and the program ends. If the player has entered 10 letters and not guessed correctly then they lose.

Write a program using pseudocode or program code for the computer game.

CONTINUED

Identify key features

Write down the inputs, processes and outputs for this system.

Inputs	Processes	Outputs
Word to guess. User's letter guess. User's word guess.	Store each letter in different index. Is user's guess letter in array? Is user's word guess correct? Count number of times the user has guessed. Repeat and stop when 10 letters or word is correct.	Letter position if it is array. 'Not in array', if not in array. 'Correct', if they guess correctly. They lose if 10 guesses and incorrect guess.

Write the inputs

```
WordToGuess ← INPUT "Enter a 5 character word"
UserLetter ← INPUT "Enter a letter to guess"
UserWord ← INPUT "Enter a word to guess"
```

Validation

The word that is input needs to be 5 characters long so that can be validated. The character entered can be validated as to whether it is a character; there is no specific way to do this but you could use a function such as `IsCharacter()`. Validating the word is not as straightforward because most things could be accepted.

```
REPEAT
    WordToGuess ← INPUT "Enter a 5 character word"
UNTIL LENGTH(WordToGuess) = 5
REPEAT
    UserLetter ← INPUT "Enter a letter to guess"
UNTIL UserLetter.IsCharacter() = TRUE

UserWord ← INPUT "Enter a word to guess"
```

Processes

Take these one at a time.

First: Store each letter in a different index in an array. To split the string you can use either a function in your language, or a pseudocode function such as substring.

```
//INPUT word to guess
REPEAT
    WordToGuess ← INPUT "Enter a 5 character word"
```

CONTINUED

```
UNTIL LENGTH(WordToGuess) = 5
//store characters in array
LetterArray ← SUBSTRING(WordToGuess, 0, 1), SUBSTRING(WordToGuess, 1, 1),
SUBSTRING(WordToGuess, 2, 1), SUBSTRING(WordToGuess, 3, 1)]

//INPUT user's guesses
REPEAT
   UserLetter ← INPUT "Enter a letter to guess"
UNTIL UserLetter.IsCharacter() = TRUE

UserWord ← INPUT "Enter a word to guess"
```

Second: Is user's guess letter in array? This will need to check all 5 values in the array and output the position if it is correct. The most efficient way to do this is to use a loop.

```
//INPUT word to guess
REPEAT
   WordToGuess ← INPUT "Enter a 5 character word"
UNTIL LENGTH(WordToGuess) = 5

//store characters in array
LetterArray ← [SUBSTRING(WordToGuess, 0, 1), SUBSTRING(WordToGuess, 1, 2),
   SUBSTRING(WordToGuess, 2, 1), SUBSTRING(WordToGuess, 3, 1)]
//INPUT user's guesses
REPEAT
   UserLetter ← INPUT "Enter a letter to guess"
UNTIL UserLetter.IsCharacter() = TRUE

//check for position of user's guess
FOR Count ← 0 TO 5
   IF UserLetter = LetterArray[Count]
      THEN
         OUTPUT "Position ", Count
   ENDIF
NEXT Count

UserWord ← INPUT "Enter a word to guess"
```

Third: Is user's word guess correct? This will involve checking the word input against the first word and outputting if it is correct.

```
LetterArray ← [SUBSTRING(WordToGuess, 0, 1), SUBSTRING(WordToGuess, 1, 1),
   SUBSTRING(WordToGuess, 2, 1), SUBSTRING(WordToGuess, 3, 1)]
//INPUT user's guesses
REPEAT
   UserLetter ← INPUT "Enter a letter to guess"
UNTIL UserLetter.IsCharacter() = TRUE
```

CONTINUED

```
//check for position of user's guess
FOR Count ← 0 TO 5
    IF UserLetter = LetterArray[Count]
        THEN
            OUTPUT "Position ", Count
    ENDIF
NEXT Count
//check user's guess
UserWord ← INPUT "Enter a word to guess"
IF UserWord = WordToGuess
    THEN
        OUTPUT "You win"
ENDIF
```

Fourth: Count number of times the user has guessed and repeat and stop when 10 letters or word is correct.

These two are being combined because they are dependent on each other. Split the processes down to:

- Repeatedly allow the user to guess the letters and word (repetition means using a loop).
- Keep a count of how many guesses they have (counting requires a variable, and incrementing this value).
- Stop when they get it correct or have had 10 guesses (the conditions for stopping the loop).

```
LetterArray ← [SUBSTRING(WordToGuess, 0, 1),
    SUBSTRING(WordToGuess, 1, 1),
    SUBSTRING(WordToGuess, 2, 1),
    SUBSTRING(WordToGuess, 3, 1)]

Count = 0

WHILE(Count < 10) REPEAT //loop for 10 guesses
    //input user's guesses
    REPEAT
        UserLetter ← INPUT "Enter a letter to guess"
    UNTIL UserLetter.IsCharacter() = TRUE

    //check for position of user's guess
    FOR Count ← 0 TO 5
        IF UserLetter = LetterArray[Count]
            THEN
                OUTPUT "Position ", Count
        ENDIF
    NEXT Count
    //check user's guess
    UserWord ← INPUT "Enter a word to guess"
```

CONTINUED

```
        IF UserWord = WordToGuess
            THEN
                OUTPUT "You win"
                Count ← 11
        ENDIF
        Count ← Count + 1
    ENDWHILE
```

Fifth: Perform the final checks:

1. Have you stored all of your inputs in variables?
2. Have you validated inputs where possible?
3. Have you performed all of the processes?
4. Have you outputted the required values?
5. Have you included appropriate messages in your outputs?
6. Have you added comments?
7. Return to the scenario and tick every statement that your program performs. If you don't tick a statement then you need to add that to your program.

Questions

1. Which elements of a program do you first need to identify?
2. What is a process in a program?
3. What do you need to include in your final checks?

11.4 Method 2: Practically carrying out the program and identifying the stages

In this method you walk through the task requirements and then turn the steps that you followed into an algorithm.

Example – calculator:

A program needs writing to act as a calculator. The program needs to:

- Take one integer value from the user.
- Take an operator as input; + for addition, − for subtraction, / for division, * for multiplication or ^ for power of.
- Take a second integer value from the user.
- Output the result.

Write a program using pseudocode or program code to solve the problem.

Perform the actions

You can do this in sequence, like in the table below. You might want to do it several times with different values:

Requirement	Run-through 1	Run-through 2	Steps followed
Take one integer value. from the user	input 10	input 2	INPUT number1
Take an operator as input; + for addition, – for subtraction, / for division, * for multiplication or ^ for power of.	input +	input ^	INPUT operator
Take a second integer value from the user.	input 20	input 3	INPUT number2
Output the result.	output 30	output 8	Check operator and output calculation.

Looking at the steps followed helps you to write the algorithm.

```
INPUT Number1
INPUT Operator
INPUT Number2
IF Operator = "+"
    THEN
        OUTPUT Number1 + Number2
ELSEIF Operator = "-"
    THEN
        OUTPUT Number1 - Number2
ELSEIF Operator = "*"
    THEN
        OUTPUT Number1 * Number2
ELSEIF Operator = "/"
    THEN
        OUTPUT Number1 / Number2
    ELSE
        OUTPUT Number1 ^ Number2
ENDIF
```

Adding extras

It is best practice to add the following to your algorithms:

- Messages to tell people what to input.
- Validation of inputs.
- Messages to say what is being output.
- Comments to explain the code.

1 Messages to tell people what to input:

```
Number1 ← INPUT "Enter the first number"
Operator ← INPUT "Enter an operator2
Number2 ← INPUT "Enter the second number"
```

```
IF Operator = "+"
    THEN
        OUTPUT Number1 + Number2
ELSEIF Operator = "-"
    THEN
        OUTPUT Number1 - Number2
ELSEIF Operator = "*"
    THEN
        OUTPUT Number1 * Number2
ELSEIF Operator = "/"
    THEN
        OUTPUT Number1 / Number2
    ELSE
        OUTPUT Number1 ^ Number2
ENDIF
```

2 Validate inputs:

```
REPEAT
    Number1 ← INPUT "Enter the first number"
UNTIL Number1.IsInteger = TRUE

REPEAT
    Operator ← INPUT "Enter an operator"
UNTIL Operator = "+" OR Operator = "-" OR Operator = "*" OR Operator = "/"
    OR Operator = "^"

REPEAT
    Number2 ← INPUT "Enter the second number"
UNTIL Number2.IsInteger = TRUE

IF Operator = "+"
    THEN
        OUTPUT Number1 + Number2
ELSEIF Operator = "-"
    THEN
        OUTPUT Number1 - Number2
ELSEIF Operator = "*"
    THEN
        OUTPUT Number1 * Number2
ELSEIF Operator = "/"
    THEN
        OUTPUT Number1 / Number2
    ELSE
        OUTPUT Number1 ^ Number2
ENDIF
```

3 Messages with outputs:

```
REPEAT
    Number1 ← INPUT "Enter the first number"
UNTIL Number1.IsInteger = TRUE
```

```
REPEAT
    Operator ← INPUT "Enter an operator"
UNTIL Operator = "+" OR Operator = "-" OR Operator = "*" OR Operator = "/"
    OR Operator = "^"

REPEAT
    Number2 ← INPUT "Enter the second number"
UNTIL Number2.IsInteger = TRUE

IF Operator = "+"
    THEN
        OUTPUT "The addition = "
        OUTPUT Number1 + Number2
ELSEIF Operator = "-"
    THEN
        OUTPUT "The subtraction = "
        OUTPUT Number1 - Number2
ELSEIF Operator = "*"
    THEN
        OUTPUT "The multiplication = "
        OUTPUT Number1 * Number2
ELSEIF Operator = "/"
    THEN
        OUTPUT "The division = "
        OUTPUT Number1 / Number2
    ELSE
        OUTPUT "The power of = "
        OUTPUT Number1 ^ Number2
ENDIF
```

4 Comments to explain the code:

```
//input the first number
REPEAT
    Number1 ← INPUT "Enter the first number"
UNTIL Number1.IsInteger = TRUE

//input the operator

REPEAT
    Operator ← INPUT "Enter an operator"
UNTIL Operator = "+" OR Operator = "-" OR Operator = "*" OR Operator = "/"
    OR Operator = "^"

//input the third number

REPEAT
    Number2 ← INPUT "Enter the second number"
UNTIL Number2.IsInteger = TRUE

//check operator and perform calculation
```

```
IF Operator = "+"
   THEN
      OUTPUT "The addition = "
      OUTPUT Number1 + Number2
ELSEIF Operator = "-"
   THEN
      OUTPUT "The subtraction = "
      OUTPUT Number1 - Number2
ELSEIF Operator = "*"
   THEN
      OUTPUT "The multiplication = "
      OUTPUT Number1 * Number2
ELSEIF Operator = "/"
   THEN
      OUTPUT "The division = "
      OUTPUT Number1 / Number2
   ELSE
      OUTPUT "The power of = "
      OUTPUT Number1 ^ Number2
ENDIF
```

SKILLS FOCUS 11.2

TACKLING A PROBLEM

In this problem an array is needed to store data within the problem. This Skills Focus will take you through the production of a solution to a problem that makes use of an array. You can then use this same methodology when solving your own problems.

A 2D array stores the cost of items and the quantity sold of 100 items. For example, part of the array could store the data in the table.

Cost	10.00	15.99	20.22	13.78	8.99	6.20	4.30	10.00	12.00
Quantity	5	6	8	10	2	15	3	16	10

A program is needed to:

- Ask the user whether they want to see the:
  - total number of items sold
  - total cost of all items sold
  - quantity of items that sold less than 10
  - quantity of items that sold more than or equal to 10.
- Calculate the answer to the item selected.
- Output the total with an appropriate message.

CONTINUED

Write an algorithm in pseudocode or program code to perform these actions.
You do not need to initialise the values in the array.

Walk through the problem

Requirement	Run-through 1	Run-through 2	Steps followed
Ask the user what they want to see.	OUTPUT "Select 1 for total number, 2 for total cost, 3 for items less than 10, 4 for items more than 10". Enter 1.	OUTPUT "Select 1 for total number, 2 for total cost, 3 for items less than 10, 4 for items more than 10". Enter 2.	OUTPUT message INPUT choice.
Calculate answer.	1 was chosen, so add up total number of items = 5 + 6 + 8 + 10 + 2 + 15 + 3 + 16 + 10	2 was chosen so calculate total cost cost*quantity for each item 10 * 5 + 15.99 + 6, etc.	If 1 was chosen total items. If 2 was chosen, multiply cost by quantity and total values. If 3 was chosen check if each value is less than 10. If 4 was chosen check if each value is more than or equal to 10.
Output answer with appropriate message.	"The total number of items sold is 75".	"The total cost is " totalCost	OUTPUT message depending on choice and total value.

Important note: this program uses an array. To check, count, or calculate using each item in an array you will need to use a loop. A count-controlled loop, e.g. for loop, is most appropriate.

Take each step and turn it into pseudocode.

Step 1 – output and input of choice.

```
OUTPUT "Enter 1 for total number of items sold, 2 for total cost of all items
    sold, 3 for number of items less than $10, 4 for number of items $10 or more".
INPUT Choice
```

Step 2 – totalling number of items sold.

```
IF Choice = 1
    THEN
        Total ← 0
        FOR Count ← 0 TO 100
            Total ← Total + Array[Count, 1]
        NEXT Count
        OUTPUT "Total number of items sold is " , Total
ENDIF
```

CONTINUED

Step 3 – totalling cost of all items sold.

```
IF Choice = 2
    THEN
        Total ← 0
        FOR Count ← 0 TO 100
            Total ← Total + (Array[Count, 0] * Array[Count, 1])
        NEXT Count
        OUTPUT "Total cost of items sold is ", Total
ENDIF
```

Step 4 – totalling number of items that sold less than 10.

```
IF Choice = 3
    THEN
        NumberItems ← 0
        FOR Count ← 0 TO 100
            IF Array[Count, 1] < 10
                THEN
                    NumberItems ← NumberItems + 1
            ENDIF
        NEXT Count
        OUTPUT "Number of items that sold less than 10 ", NumberItems
ENDIF
```

Step 5 – totalling number of items that sold 10 or more.

```
IF Choice = 4
    THEN
        NumberItems ← 0
        FOR Count ← 0 TO 100
            IF Array[Count, 1] >= 10
                THEN
                    NumberItems ← NumberItems + 1
            ENDIF
        NEXT Count
        OUTPUT "Number of items that sold 10 or more ", NumberItems
ENDIF
```

Finally combine the code. There are four selection statements that could stay as separate IF statements, but only one of these will ever run therefore using ELSEIF will be more efficient.

```
OUTPUT "Enter 1 for total number of items sold, 2 for total cost of all items
    sold, 3 for number of items less than $10, 4 for number of items $10 or
    more."
INPUT Choice
```

CONTINUED

```
IF Choice = 1
   THEN
      Total ← 0
      FOR Count ← 0 TO 100
         Total ← Total + Array[Count, 1]
      NEXT Count
      OUTPUT "Total number of items sold is " , Total
ELSEIF Choice = 2
   THEN
      Total ← 0
      FOR Count ← 0 TO 100
         Total ← Total + (Array[Count, 0] * Array[Count, 1])
      NEXT Count
      OUTPUT "Total cost of items sold is ", Total
ELSEIF Choice = 3
   THEN
      NumberItems ← 0
      FOR Count ← 0 TO 100
         IF Array[Count, 1] < 10
            THEN
               NumberItems ← NumberItems + 1
         ENDIF
      NEXT Count
      OUTPUT "Number of items that sold less than 10 ", NumberItems
ELSEIF Choice = 4
   THEN
      NumberItems ← 0
      FOR Count ← 0 to 100
         IF Array[Count, 1] >= 10
            THEN
               NumberItems ← NumberItems + 1
         ENDIF
      NEXT Count
      OUTPUT "Number of items that sold 10 or more ", NumberItems
ENDIF
```

Final checks

- Messages to tell people what to input. (✓)
- Validation of inputs – needs to be done.
- Messages to say what is being output. (✓)
- Comments to explain the code.

CONTINUED

```
//Take user choice from menu
REPEAT
    OUTPUT "Enter 1 for total number of items sold, 2 for total cost of all
        items sold, 3 for number of items less than $10, 4 for number of items
        $10 or more."
    INPUT Choice
UNTIL Choice = 1 or Choice = 2 or Choice = 3 or Choice = 4
IF Choice = 1
    THEN //if choice is 1 total number of items sold
        Total ← 0
        FOR Count ← 0 TO 100
            Total ← Total + Array[Count, 1]
        NEXT Count
        OUTPUT "Total number of items sold is " , Total
ELSEIF Choice = 2
    THEN //if choice is 2 total cost of all items sold
        Total ← 0
        FOR Count ← 0 TO 100
            Total ← Total + (Array[Count, 0] * Array[Count, 1])
        NEXT Count
        OUTPUT "Total cost of items sold is ", Total
ELSEIF Choice = 3
    THEN //if choice is 3 count items less than 10
        NumberItems ← 0
        FOR Count ← 0 TO 100
            IF Array[Count, 1] < 10
                THEN
                    NumberItems ← NumberItems + 1
            ENDIF
        NEXT Count
        OUTPUT "Number of items that sold less than 10 ", NumberItems
ELSEIF Choice = 4
    THEN //if choice is 4 total items 10 or more
        NumberItems ← 0
        FOR Count ← 0 TO 100
            IF Array[Count, 1] >= 10
                THEN
                    NumberItems ← NumberItems + 1
            ENDIF
        NEXT Count
        OUTPUT "Number of items that sold 10 or more ", NumberItems
ENDIF
```

PROGRAMMING TASKS 11.1

A character in a computer game has an x coordinate and a y coordinate, for example, x = 10, y = 20.

The current position of the player is out.

The user can then enter a command to move right, move left, move up or move down.

Moving left decreases the x coordinate by 1.

Moving right increases the x coordinate by 1.

Moving up increases the y coordinate by 1.

Moving down decreases the y coordinate by 1.

The new position of the player is output.

Getting started

1 Identify the inputs into the system.

2 Identify the processes in the system.

3 Identify the outputs from the system.

Practice

Write a program using pseudocode or program code to solve the problem.

Challenge

1 The player's x coordinate can only be between 0 and 200.
The player's y coordinate can only be between 0 and 150.

2 Add validation to your program to make sure the character's positions does not go outside these boundaries.

PROGRAMMING TASKS 11.2

A guessing game stores a number in a constant. The user has 10 attempts to try and guess the number.

If the user's guess is too high the program outputs "Too high".

If the user's guess is too low the program outputs "Too low".

If the user guesses correctly the program outputs the number of guesses they had and the game ends.

If the user has 10 guesses that are all incorrect then the game outputs the number and tells the user they have lost.

CONTINUED

Getting started

1 Identify the data that needs to be input into the system.

2 Identify the outputs that will come from the system.

3 Identify any variables or constants that you may need to use.

Practice

1 Write the program using pseudocode or program code.

2 Check that your program meets all the requirements.

3 Make sure you have added comments to explain your code.

Challenge

This task goes beyond the specification.

Programs can use random number generators. Find out how to use a random number generator and use this to declare the number at the start of the game, instead of hard coding a value into the program. Amend your program to use the random number generator.

PROGRAMMING TASKS 11.3

A program takes a 2-digit hexadecimal number as input (e.g. 3C) and outputs the binary and denary equivalent of that number.

Getting started

1 How do you convert a 2-digit hexadecimal number to binary?

2 How do you convert a 2-digit hexadecimal number to denary?

3 What inputs are needed in the system?

4 What outputs are needed in the system?

Practice

Write a program using pseudocode or program code to solve the problem.

Challenge

Extend your program to ask the user what type of data they are entering, and what type of data they want output. For example, they could enter the denary number 1 and want the 8-bit binary output 00000001.

PROGRAMMING TASKS 11.4

A program asks the user to enter the cost per kilogram for a product, e.g. $1.20.

The user then needs to enter the weight of each item while the total cost is less than $100.00.

The program should then output how many items were entered and the total cost.

Getting started

1 Identify the inputs to the system.

2 Identify the outputs to the system.

3 Identify the processes that need to take place.

Practice

Write a program using pseudocode or program code to solve the problem.

Challenge

Extend the program to store the weight and cost of each item entered in an array. When the last item is entered store the weights and costs in a text file.

PROGRAMMING TASKS 11.5

A computer program allows a user to select a username and password. The array, `Logins` , has up to 1000 indices to store the current usernames and passwords. Part of the array is shown:

Username	Rainbow	Star5	Foxes99	Purple123	Gamer55
Password	tidjsrn8	111jdk99	3jck285hkd	iii8800sndm	a45i12n3

The variable `NumberUsers` stores the how many usernames are currently stored in the array `Logins` .

A valid username is one that does not already exist in the array `Logins` .

A valid username has at least 8 characters, and at least one letter and at least one number.

Getting started

1 Identify the inputs into the system.

2 Identify the outputs from the system.

3 Identify the requirements for the username and password.

Practice

Write a program using program code or pseudocode to allow a user to enter a username and password and store this in the array `Logins`.

CONTINUED

Challenge

This task goes beyond the specification.

The usernames and passwords can be stored in a text file instead of the array. Each item can be on a different line, e.g.

username1

password1

username2

password2

Find out how to read and write multiple lines from a text file. At the start of the program read in all the usernames and passwords to an array. At the end of the program write all the usernames and passwords back into the file.

PROGRAMMING TASKS 11.6

The game rock, paper, scissors is played by two people. Each person selects rock, paper or scissors. The person who wins is determined by these rules:

- Rock wins over scissors.
- Scissors wins over paper.
- Paper wins over rock.

Getting started

1. Play the game with another student.
2. Identify the inputs in this system.
3. Identify the processes in this system.
4. Identify the outputs in this system.

Practice

1. Write a program using pseudocode or program code for the game to be played by two players.
2. Amend your program to use a function to determine which player wins. This should take the two moves as parameters and return which player won.

CONTINUED

Challenge

The second point of this challenge goes beyond the specification.

Amend your program to allow:

- The players to play 11 games and output who has won the most.
- The player to choose to play another player, or the computer. The computer will need to make use of random numbers to identify which move it should make.
- Use a file to record a high score and the player who achieved this. This should be updated, if necessary, after each game.

SUMMARY

Always read the scenario twice before starting.
Walk through the requirements, acting out each statement to identify how the system needs to work.
Identify the inputs, processes and outputs of a system.
Include validation to any inputs.
Use appropriate messages when inputting and outputting data.
Add comments to explain your code.

EXAM-STYLE QUESTIONS

1 A program takes a number from the user. It then counts from 1 to the number that the user has entered (inclusive). If the number being input is divisible by 5 it outputs a message. If the number being input is divisible by 7 it outputs a message.

Write a program using pseudocode or program code to solve the problem. **[15]**

2 A program stores the name (e.g. Dahlia), maximum height in cm (e.g. 100) and if they can live in shade (e.g. TRUE) of plants in one or more arrays.

The program asks the user to enter their requirement for a plant including:

- The minimum height required.
- If it needs to live in shade or not.

The program should then output all of the plants that meet the requirements.

Write a program using pseudocode or program code to solve the problem. You can assume the array(s) have already been initialised with 50 plants. **[15]**

CONTINUED

3 A computer game for two players uses a set of cards. Each card has an animal; horse, elephant, cat or lion.

Each player starts the game with 15 cards stored in an array; `Player1`, `Player2`.

The first card for each player is compared. The player who wins the round follows these rules:

- Lion always beats horse.
- Elephant always beats lion .
- Horse always beats cat.
- Cat always beats elephant.

The program compares each card in turn until it has compared all 16 sets of cards.

The player who wins each comparison gets 1 point. If there is a winner at the end of the game, the program should store the winner's score in a text file, and output the player's name.

Write a program using pseudocode or program code to solve the problem. You can assume the cards have already been divided between the two players. **[15]**

4 A program asks the user to enter the cost of 100 items. Each cost should be stored in an array, `Costs`. The program should calculate and output:

- The total cost of the items.
- The average cost of the item.
- The cost of the most expensive item.
- The cost of the least expensive item.

Write a program using pseudocode or program code to solve the program. **[15]**

5 A program stores words in a 1D array, `Words`, with 1000 elements.

The program needs to ask the user to select one of the three actions for the program to perform.

1 Identify and output how many words have 5 or more letters.

2 Identify and output the longest word in the array.

3 Identify and output the shortest word in the array.

The program needs to use a function for each of the three actions that are called from the main program.

Write a program using pseudocode or program code to solve the problem. You can assume the array is already initialised with 1000 words. **[15]**

SELF-EVALUATION CHECKLIST

After studying this chapter, think about how confident you are with the different topics.
This will help you to see any gaps in your knowledge and help you to learn more effectively.

You might find it helpful to rate how confident you are for each of these statements when you are revising.
You should revisit any topics that you rated 'Needs more work' or 'Getting there'.

I can...	See topic	Needs more work	Getting there	Confident to move on
identify the inputs for a system.	11.2			
identify the processes for a system.	11.2			
identify the outputs for a system.	11.2			
validate input data.	11.3			
use appropriate input and output messages.	11.3			
add comments to explain my code.	11.3			

